



机器学习-实验作业 3

李汶峰

23 级网安 1 班

202300460158

2025 年 12 月 1 日

目录

1 实验概述	2
1.1 实验目的	2
1.2 实验环境	2
1.3 实验方法	2
2 数据预处理	3
2.1 数据增强	3
2.2 数据标准化	3
3 模型构建	4
3.1 模型架构	4
3.2 模型参数量	5
4 模型训练	6
4.1 超参数设置	6
4.2 损失函数	6
4.3 优化器	7
4.4 学习率调度	7
4.5 训练流程	7
5 模型预测	8
6 结果分析与评估	9
6.1 实验结果	9
6.2 CNN 层数减半的模型评估	10
6.3 数据标准化及数据增强	12
7 附录	13

1 实验概述

1.1 实验目的

本实验旨在基于 Food-11 食物图片数据集，使用 PyTorch 框架构建卷积神经网络（CNN）模型，实现对 11 类食物的自动分类。通过设计并训练一个具有 6 个卷积块、9 层卷积的深层 CNN 模型，探索网络深度对分类性能的影响；同时实现一个层数减半但参数量相近的浅层模型，对比分析不同网络深度在相同参数规模下的特征提取能力与分类准确率差异。此外，本实验还将探究数据预处理技术（包括数据标准化 Normalization 和数据增强 Data Augmentation）对模型泛化能力的影响，通过实验验证这些技术在缓解过拟合、提升测试集准确率方面的实际效果，从而加深对 CNN 模型设计、训练策略及数据处理方法的理解与实践能力。

1.2 实验环境

- 操作系统：Windows11、Kaggle
- GPU：NVIDIA GeForce RTX 4060 GPU、Kaggle 的 P100
- 编程语言：Python
- 深度学习框架：PyTorch 2.9.1
- 主要依赖库：torch、torchvision、pandas、PIL、matplotlib
- 数据集：Food-11（训练集 9866 张、验证集 3430 张、测试集 3347 张）

1.3 实验方法

在数据预处理部分，本实验使用 Food-11 食物图片数据集，包含 11 类食物共计 16643 张图片。所有图片统一缩放至 128×128 像素，并使用 ImageNet 数据集的均值和标准差进行标准化处理，以加速模型收敛。为增强模型泛化能力，训练集采用了多种数据增强技术，包括随机裁剪缩放、随机水平翻转和随机旋转，而验证集和测试集仅进行尺寸统一和标准化处理。

模型架构方面，本实验设计了一个包含 6 个卷积块、共 9 层卷积的深层 CNN 网络。网络通道数从 64 逐步递增至 512，其中第 3、4、5 个卷积块采用双层卷积结构以增强特征提取能力。每层卷积后依次接入批归一化层和 ReLU 激活函数，并通过最大池化进行空间下采样。网络末端使用全局平均池化替代传统的全连接展平操作，有效减少了参数量。分类头部分采用两层全连接结构，并加入 Dropout 层以防止过拟合。

训练过程中，采用 AdamW 优化器进行参数更新，初始学习率设为 0.001，并配合余弦退火调度策略动态调整学习率。批大小设为 128，共训练 40 个 epoch。使用交叉熵损失函数计算分类误差，并在每个 epoch 结束后在验证集上评估模型性能，保存验证准确率最高的模型权重用于最终测试集预测。

模型训练完成后，加载验证集上表现最佳的模型权重，对测试集进行预测。将测试图片按批次送入模型，取输出 logits 中最大值对应的类别索引作为预测结果。

为探究**卷积神经网络深度**对分类性能的影响，本实验设计了一个层数减半但参数量相近的浅层模型。该模型仅包含 3 个卷积块、共 4 层卷积，通过增大每层通道数来补偿深度减少带来的参数损失，使总参数量维持在与深层模型相近的水平。其他部分与原有的深层模型保持一致，探究浅层模型的准确率变化。同时探究**数据标准化与数据增强**的影响，观察使用标准化和数据增强前后准确率的影响。

2 数据预处理

本实验的数据预处理主要包括图像尺寸统一、数据标准化和数据增强三个部分，通过 PyTorch 的 `torchvision.transforms` 模块实现。

由于原始图片尺寸不一，首先需要将所有图片统一缩放至 128×128 像素，以满足卷积神经网络对固定输入尺寸的要求。对于验证集和测试集，直接使用 `Resize` 操作；对于训练集，则通过 `RandomResizedCrop` 在裁剪的同时完成尺寸统一。

2.1 数据增强

为增强模型泛化能力、防止过拟合，仅对训练集应用以下数据增强技术：

- **随机裁剪缩放** (`RandomResizedCrop`)：随机裁剪原图 90%~100% 的区域，并缩放至 128×128
- **随机水平翻转** (`RandomHorizontalFlip`)：以 50% 的概率对图像进行左右翻转
- **随机旋转** (`RandomRotation`)：在 $[-15^\circ, +15^\circ]$ 范围内随机旋转图像

2.2 数据标准化

本次实验使用 ImageNet 数据集预计算的均值和标准差对图像进行归一化处理，将像素值从 $[0, 1]$ 区间映射到均值为 0、方差为 1 的标准分布。具体参数为：

$$\text{mean} = [0.485, 0.456, 0.406], \quad \text{std} = [0.229, 0.224, 0.225] \quad (1)$$

该操作使不同通道的数值范围趋于一致，有利于梯度稳定传播，加速模型收敛。

数据预处理通过自定义 `FoodDataset` 类实现，核心代码如下：

Listing 1: 训练集数据预处理

```
1 # 训练集数据增强
2 self.transform = transforms.Compose([
3     transforms.RandomResizedCrop(128, scale=(0.9, 1.0)),
4     transforms.RandomHorizontalFlip(),
5     transforms.RandomRotation(15),
6     transforms.ToTensor(),
7     transforms.Normalize(mean=[0.485, 0.456, 0.406],
8                             std=[0.229, 0.224, 0.225]),
9 ])
```

Listing 2: 验证集/测试集数据预处理

```
1 # 验证集/测试集预处理
2 self.transform = transforms.Compose([
3     transforms.Resize((128, 128)),
4     transforms.ToTensor(),
5     transforms.Normalize(mean=[0.485, 0.456, 0.406],
6                             std=[0.229, 0.224, 0.225]),
7 ])
```

3 模型构建

3.1 模型架构

本实验基于 PyTorch 框架构建了一个深层卷积神经网络 (CNN)，包含 6 个卷积块、共 9 层卷积，用于 11 类食物图像的分类任务。模型整体结构为：卷积特征提取层 $\rightarrow$ 全局平均池化 $\rightarrow$ 全连接分类层。输入图像尺寸为 $3 \times 128 \times 128$ (RGB 三通道)，经过 6 个卷积块逐步提取特征并降低空间分辨率，最终输出 11 维向量表示各类别的预测分数。网络结构如表 1 所示。

表 1: CNN 模型架构

层名	操作	输出尺寸	备注
Input	-	$3 \times 128 \times 128$	RGB 图像
Conv1	Conv-BN-ReLU-MaxPool	$64 \times 64 \times 64$	单层卷积
Conv2	Conv-BN-ReLU-MaxPool	$128 \times 32 \times 32$	单层卷积
Conv3	Conv-BN-ReLU-Conv-BN-ReLU-MaxPool	$256 \times 16 \times 16$	双层卷积
Conv4	Conv-BN-ReLU-Conv-BN-ReLU-MaxPool	$512 \times 8 \times 8$	双层卷积
Conv5	Conv-BN-ReLU-Conv-BN-ReLU-MaxPool	$512 \times 4 \times 4$	双层卷积
Conv6	Conv-BN-ReLU-MaxPool	$512 \times 2 \times 2$	单层卷积
GAP	AdaptiveAvgPool2d	$512 \times 1 \times 1$	全局平均池化
FC	Dropout-Linear-ReLU-Dropout-Linear	11	分类头

- **卷积块 1**: 输入 $[3, 128, 128]$ ，先通过 3×3 卷积 (通道数从 3 升维到 64, 步长 1、填充 1)，再接批归一化 (BatchNorm) 和 ReLU 激活，最后通过 2×2 最大池化将特征图下采样为 $[64, 64, 64]$ 。
- **卷积块 2**: 输入 $[64, 64, 64]$ ，卷积将通道数提升至 128，随后经过 BatchNorm、ReLU 和 2×2 最大池化，输出尺寸为 $[128, 32, 32]$ 。
- **卷积块 3**: 在特征图尺寸为 32×32 时，引入两层连续的 3×3 卷积 (通道数均为 256)，每层卷积后均接 BatchNorm 与 ReLU，以加深网络、提升特征表达能力；最后通过最大池化将空间尺寸降为 $[256, 16, 16]$ 。
- **卷积块 4**: 与卷积块 3 类似，采用两层 3×3 卷积，将通道数提升至 512，并通过最大池化将特征图压缩到 $[512, 8, 8]$ 。
- **卷积块 5**: 继续在 512 通道上进行两层卷积与池化，下采样至 $[512, 4, 4]$ 。
- **卷积块 6**: 在高维特征上再进行一次卷积与池化，将空间维度进一步压缩到 $[512, 2, 2]$ 。

核心部分代码如下：

Listing 3: 卷积层核心代码

```
1 # 卷积块 1: [3, 128, 128] -> [64, 64, 64]
2 self.conv1 = nn.Sequential(
3     nn.Conv2d(3, 64, 3, 1, 1),
4     nn.BatchNorm2d(64),
5     nn.ReLU(),
```

```
6 nn.MaxPool2d(2, 2, 0),
7 )
8
9 # 卷积块 4: [256, 16, 16] -> [512, 8, 8] (双层卷积)
10 self.conv4 = nn.Sequential(
11 nn.Conv2d(256, 512, 3, 1, 1),
12 nn.BatchNorm2d(512),
13 nn.ReLU(),
14 nn.Conv2d(512, 512, 3, 1, 1),
15 nn.BatchNorm2d(512),
16 nn.ReLU(),
17 nn.MaxPool2d(2, 2, 0),
18 )
```

卷积块输出的高维特征图尺寸为 $[512, 2, 2]$ 。为进一步降低参数量并增强对平移的鲁棒性，模型采用全局平均池化 (`AdaptiveAvgPool2d`) 将每个通道的 2×2 特征图压缩为单一标量，得到长度为 512 的特征向量。相比传统的 Flatten 接多层大规模全连接层，全局平均池化能有效减少模型参数，减轻过拟合风险。

在分类头部分，模型使用两层全连接网络完成 11 类分类：

- 首先通过一个带有 $p = 0.5$ Dropout 的线性层，将特征从 512 维映射到 256 维，并使用 ReLU 作为非线性激活；
- 随后再经过一个 $p = 0.3$ Dropout 和线性层，将 256 维特征映射到 11 维输出，对应 11 个食物类别的 logits。

Dropout 在训练过程中随机丢弃部分神经元，有助于抑制过拟合现象，从而提升模型的泛化能力。分类头部分的代码如下：

Listing 4: 分类头核心代码

```
1 # 分类头: [512] -> [11]
2 self.fc = nn.Sequential(
3 nn.Dropout(0.5),
4 nn.Linear(512, 256),
5 nn.ReLU(),
6 nn.Dropout(0.3),
7 nn.Linear(256, 11) # 输出11个类别的分数
8 )
```

3.2 模型参数量

本实验所使用的分类模型为自定义卷积神经网络 `Classifier`，包括 6 个卷积块、1 个全局平均池化层和 2 个全连接层。为分析模型复杂度，对各层的可训练参数量进行统计。

- 卷积层 Conv2d:

输入通道数为 C_{in} , 输出通道数为 C_{out} , 卷积核大小为 $K_h \times K_w$, 则参数数量为

$$\text{params}_{\text{conv}} = C_{out} \times C_{in} \times K_h \times K_w + C_{out}.$$

- 全连接层 Linear:

输入特征维度为 in , 输出特征维度为 out , 则参数数量为

$$\text{params}_{\text{fc}} = in \times out + out.$$

- 批归一化层 BatchNorm2d:

对每个通道有一个可学习的缩放系数 γ 和平移系数 β , 共 $2 \times C$ 个可训练参数:

$$\text{params}_{\text{BN}} = 2 \times C.$$

通过计算, 本次实验所使用的模型参数数量为 11720971, 如图所示:

19.9s	9	使用设备: cuda
19.9s	10	训练集路径: /kaggle/input/food
19.9s	11	训练集数量: 9866
19.9s	12	验证集数量: 3430
20.1s	13	模型参数量: 11,720,971
20.1s	14	开始训练...

图 1: 模型参数量

4 模型训练

本实验采用监督学习方式对 CNN 模型进行训练, 通过最小化分类损失函数来优化模型参数。训练过程包括前向传播、损失计算、反向传播和参数更新四个核心步骤。

4.1 超参数设置

训练过程中的主要超参数设置如表 2所示。

4.2 损失函数

采用交叉熵损失函数 (CrossEntropyLoss) 作为优化目标, 适用于多分类任务。对于 C 类分类问题, 交叉熵损失定义为:

$$\mathcal{L} = - \sum_{i=1}^C y_i \log(\hat{y}_i) \quad (2)$$

其中 y_i 为真实标签的编码, $\hat{y}_i$ 为模型预测的类别概率。

表 2: 训练超参数配置

名称	值	说明
Batch Size	128	每批处理的图片数量
Epochs	40	总训练轮数
Learning Rate	0.001	初始学习率
Weight Decay	1×10^{-5}	L2 正则化系数
Device	CUDA	使用 GPU 加速训练

4.3 优化器

选用 AdamW 优化器进行参数更新。相比传统 Adam 优化器, AdamW 将权重衰减 (Weight Decay) 从梯度更新中解耦, 直接作用于参数本身, 实现更有效的 L2 正则化效果:

$$\theta_{t+1} = \theta_t - \eta \cdot \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}} + \lambda \theta_t \right) \quad (3)$$

其中 η 为学习率, $\hat{m}_t$ 和 $\hat{v}_t$ 分别为一阶和二阶动量的偏差校正估计, λ 为权重衰减系数。

4.4 学习率调度

采用余弦退火 (Cosine Annealing) 学习率调度策略, 使学习率随训练进程平滑下降:

$$\eta_t = \eta_{min} + \frac{1}{2}(\eta_{max} - \eta_{min}) \left(1 + \cos \left(\frac{t}{T_{max}} \pi \right) \right) \quad (4)$$

其中 T_{max} 设为总训练轮数 40。该策略在训练初期保持较大学习率以快速收敛, 后期逐渐降低学习率以精细调整参数。

相应的代码如下, 首先创建数据加载器, 初始化模型、损失函数和优化器, 将模型放入 GPU 中训练, 使用交叉熵损失和 AdamW 优化器, 以及余弦退火学习率调度器, 让学习率随轮数平滑下降。

Listing 5: 初始化模型

```
1 train_loader=DataLoader(train_set,batch_size=BATCH_SIZE,shuffle=True,
   num_workers=4)
2 val_loader=DataLoader(val_set,batch_size=BATCH_SIZE,shuffle=False,
   num_workers=4)
3 model = Classifier().to(DEVICE)
4 criterion = nn.CrossEntropyLoss()
5 optimizer=torch.optim.AdamW(model.parameters(),lr=LEARNING_RATE,
   weight_decay=1e-5)
6 scheduler=torch.optim.lr_scheduler.CosineAnnealingLR(optimizer,T_max=
   N_EPOCHS)
```

4.5 训练流程

每个 Epoch 的训练流程如下:

1. **训练阶段**: 模型设为训练模式, 遍历训练集的每个 batch, 依次执行清空梯度、前向传播、计算损失、反向传播、参数更新
2. **学习率更新**: 每个 Epoch 结束后调用调度器更新学习率
3. **验证阶段**: 模型设为评估模式, 关闭梯度计算, 在验证集上评估当前模型性能
4. **模型保存**: 若当前验证准确率超过历史最优, 则保存模型权重

训练阶段过程中, 首先将数据移动到 GPU (包括图片和标签), 然后清空梯度, 将图片输入模型之中进行前向传播, 然后计算损失。获取损失后反向传播, 计算梯度并更新参数, 同时记录训练的损失和准确率。相关代码如下:

Listing 6: 训练阶段

```
1 for imgs, labels in train_loader:
2     imgs, labels = imgs.to(DEVICE), labels.to(DEVICE)
3     optimizer.zero_grad()
4     logits = model(imgs)
5     loss = criterion(logits, labels)
6     loss.backward()
7     optimizer.step()
8     train_loss += loss.item()
9     train_acc+=(logits.argmax(dim=-1)==labels).float().mean().item()
```

在每一轮训练之后, 开启评估模式, 计算当前模型对验证集的损失和准确率, 实现方法如训练过程的相似, 但是不用计算梯度, 只需要前向传播获取损失和准确率, 然后保存在验证集上准确率最高的模型。实现的代码如下:

Listing 7: 验证阶段

```
1 model.eval() # 开启评估模式
2 val_loss = val_acc = 0.0
3 with torch.no_grad(): # 不计算梯度
4     for imgs, labels in val_loader:
5         imgs, labels = imgs.to(DEVICE), labels.to(DEVICE)
6         logits = model(imgs)
7         loss = criterion(logits, labels)
8         val_loss += loss.item()
9         val_acc += (logits.argmax(dim=-1) == labels).float().mean().
            item()
```

5 模型预测

模型训练完成后, 加载验证集上表现最佳的模型权重, 对测试集进行推理预测, 并将结果保存为符合要求的 CSV 文件。模型预测的完整流程如下:

1. **加载模型**: 初始化与训练时相同结构的 CNN 模型, 并加载保存的最优权重文件

2. **设置评估模式**: 调用 `model.eval()` 关闭 Dropout 和 BatchNorm 的训练行为, 确保推理结果的确定性
3. **批量推理**: 将测试集图片按批次送入模型, 获取每张图片的预测类别
4. **结果整理**: 将图片 ID 与预测类别配对, 按 ID 升序排列后导出为 CSV 文件

预测部分的核心代码如下, 首先先获取测试集的加载器并将数据放入 GPU 中, 然后开启评估模式, 使用无梯度的循环, 将图片输入到模型之中然后选择所有类别中概率最高的作为最终输出。

Listing 8: 预测阶段

```
1 test_loader = DataLoader(test_set, batch_size=BATCH_SIZE, shuffle=False
    , num_workers=4)
2 model = Classifier().to(DEVICE)
3 model.load_state_dict(torch.load(MODEL_PATH))
4 model.eval()
5 predictions, ids = [], []
6 with torch.no_grad():
7     for imgs, filenames in test_loader:
8         logits = model(imgs.to(DEVICE))
9         predictions.extend(logits.argmax(dim=-1).cpu().numpy())
```

6 结果分析与评估

6.1 实验结果

实验最终运行结果如图所示, 在运行 40 轮的循环之后, 得出的结果为:

- 训练参数量: 11,720,971
- 验证集准确率为: 0.7624
- 验证集损失为: 0.8402
- 训练集准确率: 0.8935
- 训练集损失: 0.3287

1153.9s	76	[Epoch 37/40] 训练集 Loss: 0.3381, Acc: 0.8866 验证集 Loss: 0.8418, Acc: 0.7589
1153.9s	77	✓ 保存最佳模型 (验证集准确率: 0.7589)
1183.5s	78	[Epoch 38/40] 训练集 Loss: 0.3219, Acc: 0.8950 验证集 Loss: 0.8312, Acc: 0.7621
1183.5s	79	✓ 保存最佳模型 (验证集准确率: 0.7621)
1213.4s	80	[Epoch 39/40] 训练集 Loss: 0.3341, Acc: 0.8873 验证集 Loss: 0.8402, Acc: 0.7624
1213.4s	81	✓ 保存最佳模型 (验证集准确率: 0.7624)
1243.7s	82	[Epoch 40/40] 训练集 Loss: 0.3287, Acc: 0.8935 验证集 Loss: 0.8342, Acc: 0.7607
1243.7s	83	
1243.7s	84	训练完成! 最佳验证集准确率: 0.7624

图 2: 运行结果图

本实验将代码部署至 Kaggle 平台运行，利用其提供的 GPU 算力资源加速模型训练。实验采用上述设计的 6 层卷积块、共 9 层卷积的深层 CNN 模型进行食物图像分类任务。在数据预处理阶段，对训练集实施了多种数据增强策略，以扩充训练样本的多样性、增强模型的泛化能力。同时，对所有数据集均采用基于 ImageNet 统计量的标准化处理。训练阶段采用 AdamW 优化器，初始学习率设为 0.001，并配合余弦退火学习率调度策略动态调整学习率。批大小设置为 128，共训练 40 个 Epoch。训练过程中同步记录每轮的训练损失、训练准确率、验证损失和验证准确率，训练完成后生成的损失曲线和准确率曲线如图所示：

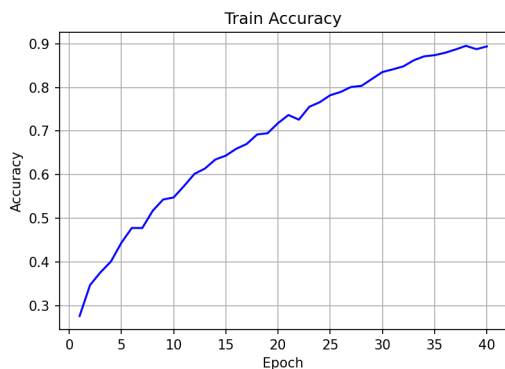


图 3: 训练集准确率

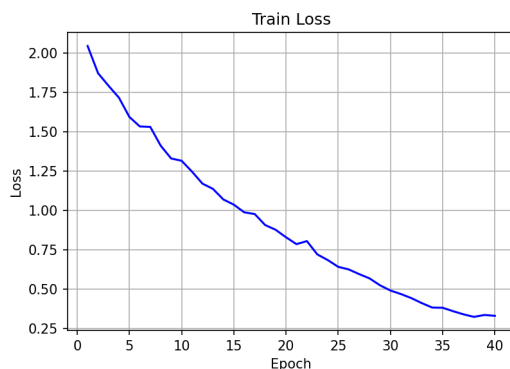


图 4: 训练集损失变化

在训练过程中也对验证集进行了验证，并记录了模型在验证集上的损失和准确率，实验结果如下图所示。最终的实验结果为：在 40 轮的训练后，验证集的损失降低到了 0.8342，最高准确率为 0.7624。

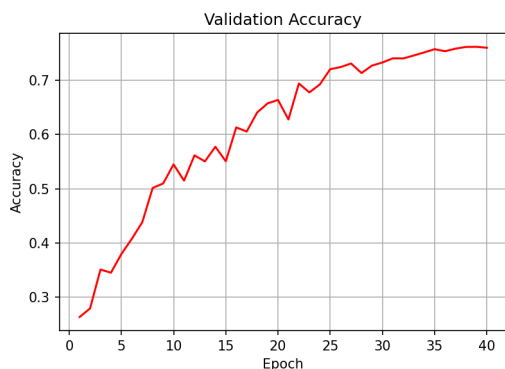


图 5: 验证集准确率

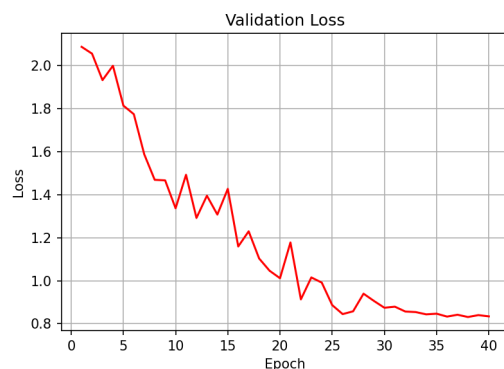


图 6: 验证集损失变化

6.2 CNN 层数减半的模型评估

本题的目标是在与第一题深层模型参数量大致相近的前提下，将卷积神经网络的深度（卷积层数）约减半，从而分析模型深度对分类性能的影响。在原来的模型中，使用了 6 层卷积块、共 9 层卷积的深层 CNN 模型。为实现“深度减半、参数量接近”的目标，构建了浅层的 CNN 模型，仅保留 4 个卷积块、共 4 层卷积，每个卷积块由一层 3×3 卷积、BatchNorm、ReLU 激活以及 2×2 最大池化构成，但是每个卷积块的通道数显著加宽，以使得参数量与原来相近。其具体设计如下：

- 卷积块 1: $[3, 128, 128] \rightarrow [192, 64, 64]$;

- 卷积块 2: $[192, 64, 64] \rightarrow [384, 32, 32]$;
- 卷积块 3: $[384, 32, 32] \rightarrow [768, 16, 16]$;
- 卷积块 4: $[768, 16, 16] \rightarrow [1024, 8, 8]$;
- 分类头采用: $1024 \rightarrow 512 \rightarrow 11$
- 总参数量为: 10938379

实验结果如图所示:

```
1705.3s   70      ✓ 保存最佳模型 (验证集准确率: 0.7284)
1752.3s   71      [Epoch 36/40] 训练集 Loss: 0.6800, Acc: 0.7725 | 验证集 Loss: 0.8348, Acc: 0.7202
1799.2s   72      [Epoch 37/40] 训练集 Loss: 0.6587, Acc: 0.7699 | 验证集 Loss: 0.8211, Acc: 0.7256
1846.9s   73      [Epoch 38/40] 训练集 Loss: 0.6726, Acc: 0.7715 | 验证集 Loss: 0.8117, Acc: 0.7333
1846.9s   74      ✓ 保存最佳模型 (验证集准确率: 0.7333)
1893.8s   75      [Epoch 39/40] 训练集 Loss: 0.6398, Acc: 0.7804 | 验证集 Loss: 0.8242, Acc: 0.7234
1940.9s   76      [Epoch 40/40] 训练集 Loss: 0.6589, Acc: 0.7807 | 验证集 Loss: 0.8275, Acc: 0.7243
1940.9s   77
1940.9s   78      训练完成! 最佳验证集准确率: 0.7333
```

图 7: 层数减半后的运行结果图

从上图可以看出, 在层数减半的情况最高验证集准确率为: 0.7333, 相比于深层网络的准确率 0.7624 有所降低。而训练集在训练 40 轮后训练集准确率 0.7807 也比深层网络的 0.8935 较低。

在控制总参数量大致相同的条件下, 将 CNN 的卷积层数从 9 层减为 4 层, 并通过加宽通道数与全连接层来补偿参数量, 实验结果表明: 浅层宽网络在训练集上的拟合能力比深层网络低, 且在验证集上的最终准确率也较低, 说明在图像分类任务中, 适当的网络深度在参数量相近的情况下仍然能够带来更好的特征表达与泛化性能; 仅仅增加宽度而减少深度并不能完全替代深层结构所提供的层级表示能力, 在一定范围内层数越多训练拟合效果就越好。

- 训练参数量: 11,720,971
- 最高验证集准确率为: 0.7333
- 验证集损失为: 0.8275
- 训练集准确率: 0.7807
- 训练集损失: 0.6589

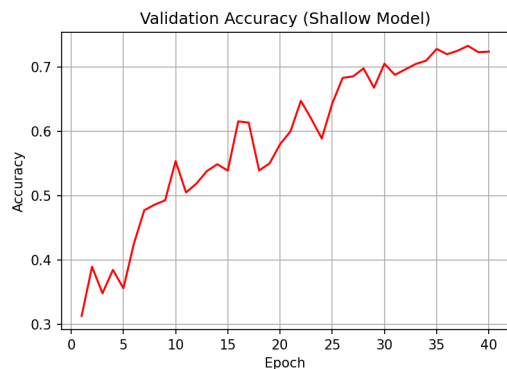


图 8: 层数减半验证集准确率

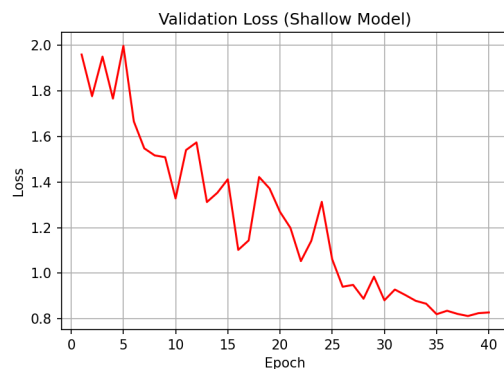


图 9: 层数减半验证集损失

6.3 数据标准化及数据增强

在之前的实验中，我们使用了多种数据增强的方法，包括随机裁剪缩放、随机水平翻转、随机旋转三个部分。也使用了 ImageNet 数据集预计算的均值和标准差对图像进行归一化处理，进行了数据标准化。现在需要去除数据标准化和数据增强，探究其对准确率的影响。实验的结果如图所示

```

1018.6s 68 [Epoch 36/40] 训练集 Loss: 0.0894, Acc: 0.9732 | 验证集 Loss: 1.1565, Acc: 0.7473
1018.6s 69 ✓ 保存最佳模型 (验证集准确率: 0.7473)
1045.5s 70 [Epoch 37/40] 训练集 Loss: 0.0738, Acc: 0.9800 | 验证集 Loss: 1.2018, Acc: 0.7467
1072.6s 71 [Epoch 38/40] 训练集 Loss: 0.0791, Acc: 0.9800 | 验证集 Loss: 1.1875, Acc: 0.7457
1099.4s 72 [Epoch 39/40] 训练集 Loss: 0.0755, Acc: 0.9799 | 验证集 Loss: 1.1922, Acc: 0.7460
1126.5s 73 [Epoch 40/40] 训练集 Loss: 0.0724, Acc: 0.9780 | 验证集 Loss: 1.1971, Acc: 0.7469
1126.5s 74
1126.5s 75 训练完成! 最佳验证集准确率: 0.7473
    
```

图 10: 去除数据标准化及数据增强后的运行结果图

通过上述结果可以看出变化，模型在测试集上的准确率有所降低为 0.7473，损失也有所增高（相比于使用了标准化和数据增强前的准确率 0.7624）。但是对于训练集上却拟合得很好，在 40 轮训练后，在训练集上的准确率高达 0.9780，具有严重的过拟合现象。这说明了虽然数据标准化和数据增强虽然对于训练集的拟合难度有所增加，训练准确率较低，但是验证集的准确率却有所增加，这表明数据增强有效缓解了过拟合，显著提高了模型的泛化能力。

- 最高验证集准确率为: 0.7473
- 验证集损失为: 1.1971
- 训练集准确率: 0.9780
- 训练集损失: 0.0724

同时画出验证集上的准确率和损失曲线图，如图所示，损失下降和准确率上升均波动较大，这是因为在去除数据增强及标准化后模型容易过拟合到训练集的偶然噪声，从而使损失下降和准确率上升的曲线在不同 epoch 之间呈现出更大的波动。

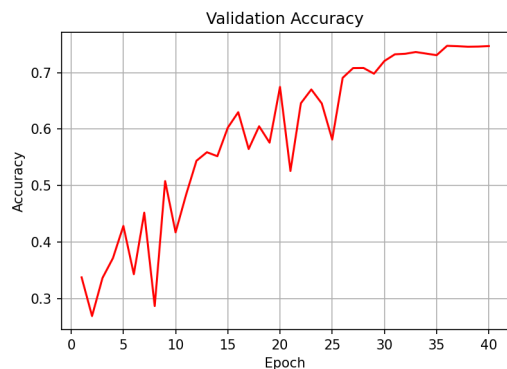


图 11: 去除数据增强及标准化后验证集准确率

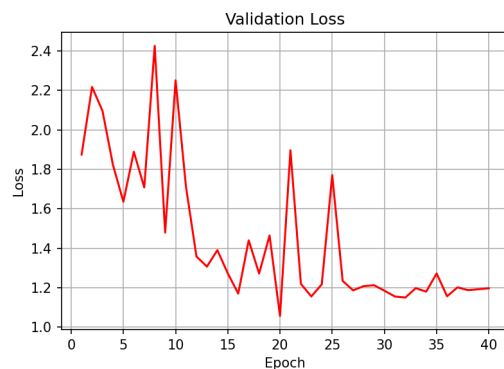


图 12: 去除数据增强及标准化后验证集损失

总结: 数据标准化对模型性能提升方面发挥了积极作用, 而数据增强增强了模型的泛化能力防止了过拟合, 同时也提高了模型的性能。

7 附录

- dataset.py: 数据加载文件, 用于加载食物图片
- model.py: 模型代码文件, 定义了图像分类器的模型
- model2.py: 层数减半后的模型代码
- predict.py: 图片预测文件, 用于对测试集进行预测
- train.py: 训练代码文件, 用于训练模型
- prediction.csv: 预测结果文件